Operator Overloading in Python

Operator overloading in Python allows us to define how built-in operators behave when used with user-defined objects. This is done by defining special methods (also known as **magic methods** or **dunder methods**) in a class.

Operator Overloading Restrictions

There are some restrictions on operator overloading:

• The precedence of an operator cannot be changed by overloading. However, parentheses can be used to force evaluation order in an expression.
• The left-to-right or right-to-left grouping of an operator cannot be changed by overloading.
• The “arity” of an operator—that is, whether it’s a unary or binary operator—connote changed.
• You cannot create new operators—only existing operators can be overloaded.
• The meaning of how an operator works on objects of built-in types cannot be changed. You cannot, for example, change + so that it subtracts two integers.
• Operator overloading works only with objects of custom classes or with a mixture of an object of a custom class and an object of a built-in type.

For example, the + operator is typically used for addition, but we can overload it to perform concatenation or custom addition in a user-defined class.

Program 1:

```
# complexnumber.py
"""Complex class with overloaded +, +=, -, -=, *, *=, /, == operators."""

class Complex:
    """Complex class that represents a complex number
    with real and imaginary parts."""

    def __init__(self, real, imaginary):
        """Initialize Complex class attributes."""
        self.real = real
        self.imaginary = imaginary

    # --------------------------
    # Addition
    # --------------------------
    def __add__(self, right):
        return Complex(self.real + right.real,
                       self.imaginary + right.imaginary)

    def __iadd__(self, right):
        self.real += right.real
        self.imaginary += right.imaginary
        return self
```

```
# --------------------------
# Subtraction
# --------------------------
def __sub__(self, right):
    return Complex(self.real - right.real,
                   self.imaginary - right.imaginary)

def __isub__(self, right):
    self.real -= right.real
    self.imaginary -= right.imaginary
    return self

# --------------------------
# Multiplication
# (a+bi)(c+di) = (ac - bd) + (ad + bc)i
# --------------------------
def __mul__(self, right):
    real_part = self.real * right.real - self.imaginary * right.imaginary
    imag_part = self.real * right.imaginary + self.imaginary * right.real
    return Complex(real_part, imag_part)

def __imul__(self, right):
    real_part = self.real * right.real - self.imaginary * right.imaginary
    imag_part = self.real * right.imaginary + self.imaginary * right.real
    self.real = real_part
    self.imaginary = imag_part
    return self

# --------------------------
# Division
# --------------------------
def __truediv__(self, right):
    denominator = right.real**2 + right.imaginary**2

    if denominator == 0:
        raise ZeroDivisionError("Cannot divide by zero")

    real_part = (self.real * right.real + self.imaginary * right.imaginary) / denominator
    imag_part = (self.imaginary * right.real - self.real * right.imaginary) / denominator

    return Complex(real_part, imag_part)

# --------------------------
# Equality
# --------------------------
def __eq__(self, right):
    return self.real == right.real and self.imaginary == right.imaginary

# --------------------------
```

```
    # String Representation
    # --------------------------
    def __repr__(self):
        sign = '+' if self.imaginary >= 0 else '-'
        return f'({self.real} {sign} {abs(self.imaginary)}i)'


# --------------------------
# Testing the operators
# --------------------------

x = Complex(2, 4)
y = Complex(5, -1)

print("x =", x)
print("y =", y)

# Test +
print("\nAddition:")
print("x + y =", x + y)

# Test +=
x += y
print("After x += y, x =", x)

# Test -
a = Complex(10, 6)
b = Complex(3, 2)

print("\nSubtraction:")
print("a - b =", a - b)

# Test -=
a -= b
print("After a -= b, a =", a)

# Test *
print("\nMultiplication:")
print("a * b =", a * b)

# Test *=
a *= b
print("After a *= b, a =", a)

# Test /
c = Complex(4, 2)
d = Complex(1, -1)
print("\nDivision:")
print("c / d =", c / d)
```

# Test ==
e = Complex(3, 4)
f = Complex(3, 4)
print("\nEquality:")
print("e == f:", e == f)

note: __add__ returns a new object without modifying operands.

__iadd__ modifies the left operand and returns it.

output:-

```
x = (2 + 4i)
y = (5 - 1i)

Addition:
x + y = (7 + 3i)
After x += y, x = (7 + 3i)

Subtraction:
a - b = (7 + 4i)
After a -= b, a = (7 + 4i)

Multiplication:
a * b = (13 + 26i)
After a *= b, a = (13 + 26i)

Division:
c / d = (1.0 + 3.0i)

Equality:
e == f: True
```

program 2: inheritance hierarchy

```
from decimal import Decimal

class CommissionEmployee:
    """An employee who gets paid commission based on gross sales."""

    def __init__(self, first_name, last_name, ssn, gross_sales,
commission_rate):
        """Initialize CommissionEmployee's attributes."""
        self._first_name = first_name
        self._last_name = last_name
        self._ssn = ssn
        self.gross_sales = gross_sales  # Validate via property
        self.commission_rate = commission_rate  # Validate via property

    @property
    def first_name(self):
```

```
        return self._first_name

    @property
    def last_name(self):
        return self._last_name

    @property
    def ssn(self):
        return self._ssn

    @property
    def gross_sales(self):
        return self._gross_sales

    @gross_sales.setter
    def gross_sales(self, sales):
        """Set gross sales or raise ValueError if invalid."""
        if sales < Decimal('0.00'):
            raise ValueError('Gross sales must be >= 0')
        self._gross_sales = sales

    @property
    def commission_rate(self):
        return self._commission_rate

    @commission_rate.setter
    def commission_rate(self, rate):
        """Set commission rate or raise ValueError if invalid."""
        if not (Decimal('0.0') < rate < Decimal('1.0')):
            raise ValueError('Commission rate must be greater than 0
and less than 1')
        self._commission_rate = rate

    def earnings(self):
        """Calculate earnings."""
        return self.gross_sales * self.commission_rate

    def __repr__(self):
        """Return string representation for repr()."""
        return (f'CommissionEmployee: {self.first_name}
{self.last_name}\n'
                f'Social Security Number: {self.ssn}\n'
                f'Gross Sales: {self.gross_sales:.2f}\n'
                f'Commission Rate: {self.commission_rate:.2f}')


class SalariedCommissionEmployee(CommissionEmployee):
    """An employee who gets paid a salary plus commission based on
gross sales."""
```

```
    def __init__(self, first_name, last_name, ssn, gross_sales,
commission_rate, base_salary):
        """Initialize SalariedCommissionEmployee's attributes."""
        super().__init__(first_name, last_name, ssn, gross_sales,
commission_rate)
        self.base_salary = base_salary  # Validate via property

    @property
    def base_salary(self):
        return self._base_salary

    @base_salary.setter
    def base_salary(self, salary):
        """Set base salary or raise ValueError if invalid."""
        if salary < Decimal('0.00'):
            raise ValueError('Base salary must be >= 0')
        self._base_salary = salary

    def earnings(self):
        """Calculate earnings."""
        return super().earnings() + self.base_salary

    def __repr__(self):
        """Return string representation for repr()."""
        return (f'Salaried {super().__repr__()}\n'
                f'Base Salary: {self.base_salary:.2f}')


# Testing the functionality
if __name__ == "__main__":
    # Creating a CommissionEmployee instance
    c = CommissionEmployee('Sue', 'Jones', '333-33-3333',
Decimal('10000.00'), Decimal('0.06'))
    print(c)
    print(f'Earnings: {c.earnings():,.2f}\n')

    # Modifying attributes
    c.gross_sales = Decimal('20000.00')
    c.commission_rate = Decimal('0.1')
    print(c)
    print(f'Updated Earnings: {c.earnings():,.2f}\n')

    # Creating a SalariedCommissionEmployee instance
    s = SalariedCommissionEmployee('Bob', 'Lewis', '444-44-4444',
Decimal('5000.00'), Decimal('0.04'), Decimal('300.00'))
    print(s)
    print(f'Earnings: {s.earnings():,.2f}\n')
```

```
    # Modifying attributes
    s.gross_sales = Decimal('10000.00')
    s.commission_rate = Decimal('0.05')
    s.base_salary = Decimal('1000.00')
    print(s)
    print(f'Updated Earnings: {s.earnings():,.2f}\n')

    # Checking inheritance relationships
    print(issubclass(SalariedCommissionEmployee,
CommissionEmployee))  # True
    print(isinstance(c, CommissionEmployee))  # True
    print(isinstance(s, SalariedCommissionEmployee))  # True
```

```
CommissionEmployee: Sue Jones
Social Security Number: 333-33-3333
Gross Sales: 10000.00
Commission Rate: 0.06
Earnings: 600.00

CommissionEmployee: Sue Jones
Social Security Number: 333-33-3333
Gross Sales: 20000.00
Commission Rate: 0.10
Updated Earnings: 2,000.00

Salaried CommissionEmployee: Bob Lewis
Social Security Number: 444-44-4444
Gross Sales: 5000.00
Commission Rate: 0.04
Base Salary: 300.00
Earnings: 500.00

Salaried CommissionEmployee: Bob Lewis
Social Security Number: 444-44-4444
Gross Sales: 10000.00
Commission Rate: 0.05
Base Salary: 1000.00
Updated Earnings: 1,500.00

True
True
True
```

Testing the "is a" Relationship

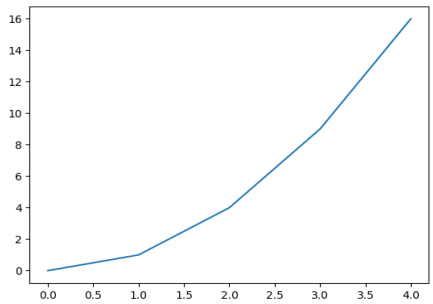
Python provides two built-in functions—**issubclass** and **isinstance**—for testing "is a" relationships. Function issubclass determines whether one class is derived from another: Function isinstance determines whether an object has an "is a" relationship with a specific type. Because SalariedCommissionEmployee inherits from CommissionEmployee, both of the following snippets return True, confirming the "is a" relationship.

Matplotlib

Matplotlib is a powerful and versatile open-source plotting library for Python, designed to help users visualize data in a variety of formats. Developed by John D. Hunter in 2003, it enables users to graphically represent data, facilitating easier analysis and understanding.

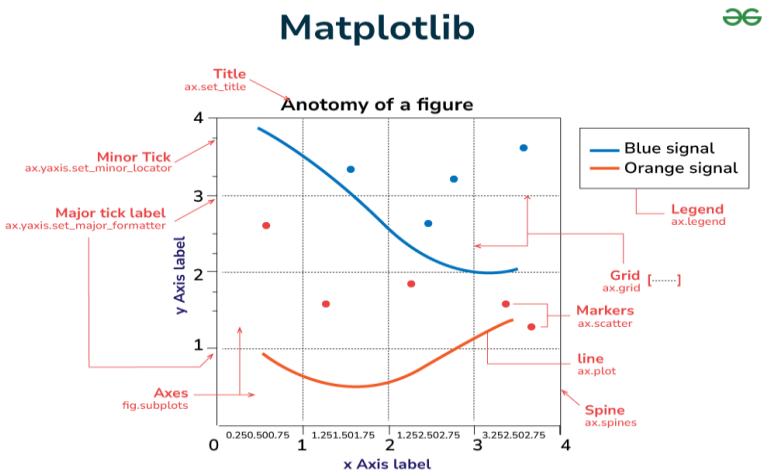
Example of a Plot in Matplotlib:

```
import matplotlib.pyplot as plt

x = [0, 1, 2, 3, 4]
y = [0, 1, 4, 9, 16]

plt.plot(x, y)
plt.show()
```



Components or Parts of Matplotlib Figure

Anatomy of a Matplotlib Plot: This section dives into the key components of a Matplotlib plot, including figures, axes, titles, and legends, essential for effective data visualization.



The parts of a Matplotlib figure include (as shown in the figure above):

- **Figure**: The overarching container that holds all plot elements, acting as the canvas for visualizations.
- **Axes**: The areas within the figure where data is plotted; each figure can contain multiple axes.

- **Axis**: Represents the x-axis and y-axis, defining limits, tick locations, and labels for data interpretation.
- **Lines and Markers**: Lines connect data points to show trends, while markers denote individual data points in plots like scatter plots.
- **Title and Labels**: The title provides context for the plot, while axis labels describe what data is being represented on each axis.

Marker properties:

We can use marker property to highlight data points on the line plot.

- plt.plot(x,y,marker='.')
- The allowed markers are :
- . -point markers
- O –circle
- V-tringle down
- ^-tringle up
- D-dimond
- +- plus marker
- s(small s)-square marker
- | - tri-down marker

Linestyle Properties

Line style specifies whether the line is solid or dashed or dotted etc.
We can specify line style by using linestyle keyword argument.

- - : solid line
- --:dashed line
- : -> dotted line
- -. ->dashed-doted line
- plt.plot(x,y,marker='o', linestyle=':')

Color Properties

We can specify our required color for line plot.
We have to use any color keyword argument.
We can use any color even hexa code also like '#A7E30E'.
Matplotlib defines some short code for different color.

- b- blue
- g- green
- r- red
- c- cyan
- m- magenta
- y-yellow
- k-black
- w –white
- Orange has full name no short name

Plotting the datapoints

```
import pandas as pd
```

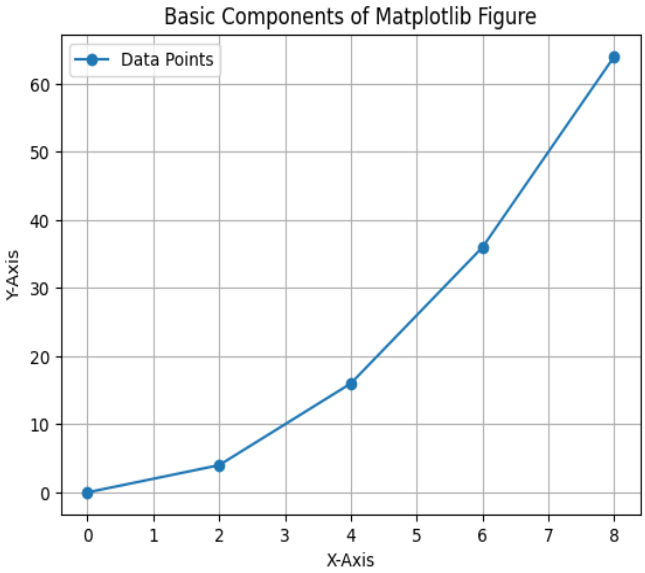
```
import matplotlib.pyplot as plt

# Lambda function for Fahrenheit to Celsius conversion
celsius = lambda f: 5 / 9 * (f - 32)

# Creating a list of (Fahrenheit, Celsius) tuples for 0 to 100°F in 10-degree increments
temps = [(f, celsius(f)) for f in range(0, 101, 10)]

# Creating a DataFrame
temps_df = pd.DataFrame(temps, columns=['Fahrenheit', 'Celsius'])

# Plotting the data points
axes = temps_df.plot(x='Fahrenheit', y='Celsius', style='.-', legend=True)

# Setting y-axis label
axes.set_ylabel('Celsius')

# Display the plot
plt.show()
```

Matplotlib Pyplot

Pyplot is a module within Matplotlib that provides a MATLAB-like interface for making plots. It simplifies the process of adding plot elements such as lines, images, and text to the axes of the current figure. **Steps to Use Pyplot:**

- **Import Matplotlib:** Start by importing matplotlib.pyplot as plt.
- **Create Data:** Prepare your data in the form of lists or arrays.
- **Plot Data**: Use plt.plot() to create the plot.
- **Customize Plot**: Add titles, labels, and other elements using methods like plt.title(), plt.xlabel(), and plt.ylabel().
- **Display Plot**: Use plt.show() to display the plot.

```
import matplotlib.pyplot as plt

x = [0, 2, 4, 6, 8]
y = [0, 4, 16, 36, 64]

fig, ax = plt.subplots()
ax.plot(x, y, marker='o', label="Data Points")

ax.set_title("Basic Components of Matplotlib Figure")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")


plt.show()
```



Different Types of Plots in Matplotlib

Matplotlib offers a wide range of plot types to suit various data visualization needs. Here are some of the most commonly used types of plots in Matplotlib:

- 1. Line Graph
- 2. Bar Chart
- 3. Histogram
- 4. Scatter Plot
- 5. Pie Chart
- 6. 3D Plot

Bar Chart

'''

Bar chart-A bar chart is a type of graph that represents categorical data with rectangular bars. The length or height of each bar corresponds to the frequency or numerical value of the category it represents. Bar charts are used to compare different categories or to show the distribution of data across categories.

In a typical bar chart:

The x-axis represents the categories being compared.
The y-axis represents the frequency, count, or value associated with eachcategory.Each bar represents a specific category, with the height or length of the bar proportional to the value it represents.

'''

```
from matplotlib import pyplot as plt

# Movie titles
movies = ["Annie Hall", "Ben-Hur", "Casablanca", "Gandhi", "West Side Story"]

# Number of Oscars won by each movie
num_oscars = [5, 11, 3, 8, 10]
```

```
# Plot bars with left x-coordinates [0, 1, 2, 3, 4], heights [num_oscars]
plt.bar(range(len(movies)), num_oscars)

# Add a title
plt.title("My Favorite Movies")

# Add a label to the y-axis
plt.ylabel("# of Academy Awards")

# Label x-axis with movie names at bar centers
plt.xticks(range(len(movies)), movies)

# Show the plot
plt.show()
```

Line Graph

```
'''
Line charts- A line chart is a type of graphical representation used to
visualize data trends over time. It consists of a series of data points
plotted on a graph, with lines connecting the points to show the progressionor trend of the data. Line charts are particularly useful for displayingcontinuous data sets and identifying patterns or changes in the data over time.
'''

from matplotlib import pyplot as plt

years = [1950, 1960, 1970, 1980, 1990, 2000, 2010]
gdp = [300.2, 543.3, 1075.9, 2862.5, 5979.6, 10289.7, 14958.3]

# Create a line chart with years on the x-axis and GDP on the y-axis
plt.plot(years, gdp, color='green', marker='o', linestyle='solid')

# Add a title
plt.title("Nominal GDP")

# Add a label to the y-axis
plt.ylabel("Billions of $")

# Add a label to the x-axis
plt.xlabel("Year")

# Show the plot
plt.show()

# Save the plot
plt.savefig("gdp_plot.png")
```

HISTOGRAM

'''
A histogram is a graphical representation of the distribution of numerical data. It consists of a set of rectangles whose area is proportional to the frequency of the data they represent. Histograms are commonly used to visualize the distribution of continuous data, such as measurements or observations.
'''

```
from collections import Counter
from matplotlib import pyplot as plt

# Grades data
grades = [83, 95, 91, 87, 70, 0, 85, 82, 100, 67, 73, 77, 0]

# Bucket grades by decile, but put 100 in with the 90s
histogram = Counter(min(grade // 10 * 10, 90) for grade in grades)

# Create a bar plot
plt.bar([x + 5 for x in histogram.keys()],  # Shift bars right by 5
        histogram.values(),                 # Give each bar its correct height
        10,                                 # Give each bar a width of 10
        edgecolor=(0, 0, 0))                # Black edges for each bar

# Set x-axis limits
plt.axis([-5, 105, 0, 5])  # x-axis from -5 to 105, y-axis from 0 to 5

# Set x-axis ticks
plt.xticks([10 * i for i in range(11)])  # x-axis labels at 0, 10,...,100

# Label x and y axes
plt.xlabel("Decile")
plt.ylabel("# of Students")

# Add a title
plt.title("Distribution of Exam 1 Grades")

# Show the plot
plt.show()
```

Case Study: Card Shuffling and Dealing Simulation

"""
Complete Card Shuffling and Dealing Simulation
Single-file version
"""

```
import random
from pathlib import Path
import matplotlib.pyplot as plt
import matplotlib.image as mpimg


# ---------------------------------------------
# Card Class
# ---------------------------------------------
class Card:
    # Class attributes (shared by all cards)
    FACES = ['Ace', '2', '3', '4', '5', '6',
             '7', '8', '9', '10', 'Jack', 'Queen', 'King']

    SUITS = ['Hearts', 'Diamonds', 'Clubs', 'Spades']

    def __init__(self, face, suit):
        self._face = face
        self._suit = suit

    @property
    def face(self):
        return self._face

    @property
    def suit(self):
        return self._suit

    @property
    def image_name(self):
        return str(self).replace(' ', '_') + '.png'

    def __repr__(self):
        return f"Card(face='{self.face}', suit='{self.suit}')"

    def __str__(self):
        return f"{self.face} of {self.suit}"

    def __format__(self, format_spec):
        return f"{str(self):{format_spec}}"


# ---------------------------------------------
# DeckOfCards Class
# ---------------------------------------------
class DeckOfCards:
    NUMBER_OF_CARDS = 52

    def __init__(self):
        self._current_card = 0
```

```
        self._deck = []

        # Create 52 cards in order
        for count in range(DeckOfCards.NUMBER_OF_CARDS):
            self._deck.append(
                Card(Card.FACES[count % 13],
                     Card.SUITS[count // 13])
            )

    def shuffle(self):
        self._current_card = 0
        random.shuffle(self._deck)

    def deal_card(self):
        try:
            card = self._deck[self._current_card]
            self._current_card += 1
            return card
        except IndexError:
            return None

    def __str__(self):
        s = ''
        for index, card in enumerate(self._deck):
            s += f"{card:<19}"
            if (index + 1) % 4 == 0:
                s += '\n'
        return s


# ---------------------------------------------
# Main Program
# ---------------------------------------------
if __name__ == "__main__":

    # Create deck
    deck_of_cards = DeckOfCards()

    print("ORIGINAL DECK\n")
    print(deck_of_cards)

    # Shuffle deck
    deck_of_cards.shuffle()
    print("\nSHUFFLED DECK\n")
    print(deck_of_cards)

    # Deal 5 cards
    print("\nDEALING 5 CARDS")
    for _ in range(5):
        print(deck_of_cards.deal_card())
```

```
# ------------------------------------------------
# OPTIONAL: Display Card Images (requires images)
# ------------------------------------------------
"""
Make sure:
- You have a folder named 'card_images'
- It contains 52 PNG files
- Names like: Ace_of_Hearts.png, 10_of_Spades.png etc.
"""

try:
    print("\nDisplaying card images...")

    deck_of_cards.shuffle()

    path = Path('.').joinpath('card_images')
    figure, axes_list = plt.subplots(nrows=4, ncols=13)

    for axes in axes_list.ravel():
        axes.get_xaxis().set_visible(False)
        axes.get_yaxis().set_visible(False)

        card = deck_of_cards.deal_card()
        image_name = card.image_name

        img = mpimg.imread(str(path.joinpath(image_name).resolve()))
        axes.imshow(img)

    figure.tight_layout()
    plt.show()

except Exception as e:
    print("\nImage display skipped (card_images folder not found).")
    print("Error:", e)
```

```
ORIGINAL DECK

Ace of Hearts      2 of Hearts        3 of Hearts        4 of Hearts
5 of Hearts        6 of Hearts        7 of Hearts        8 of Hearts
9 of Hearts        10 of Hearts       Jack of Hearts     Queen of Hearts
King of Hearts     Ace of Diamonds    2 of Diamonds      3 of Diamonds
4 of Diamonds      5 of Diamonds      6 of Diamonds      7 of Diamonds
8 of Diamonds      9 of Diamonds      10 of Diamonds     Jack of Diamonds
Queen of Diamonds  King of Diamonds   Ace of Clubs       2 of Clubs
3 of Clubs         4 of Clubs         5 of Clubs         6 of Clubs
7 of Clubs         8 of Clubs         9 of Clubs         10 of Clubs
Jack of Clubs      Queen of Clubs     King of Clubs      Ace of Spades
```

```
2 of Spades        3 of Spades        4 of Spades        5 of Spades
6 of Spades        7 of Spades        8 of Spades        9 of Spades
10 of Spades       Jack of Spades     Queen of Spades    King of Spades


SHUFFLED DECK

Jack of Hearts     Queen of Hearts    Queen of Spades    4 of Diamonds
Queen of Diamonds  5 of Hearts        5 of Diamonds      10 of Hearts
5 of Clubs         4 of Spades        5 of Spades        6 of Clubs
8 of Diamonds      King of Clubs      2 of Clubs         10 of Clubs
6 of Spades        8 of Clubs         10 of Diamonds     Queen of Clubs
9 of Hearts        4 of Hearts        9 of Spades        9 of Clubs
3 of Spades        Jack of Clubs      Jack of Diamonds   2 of Hearts
3 of Clubs         Ace of Spades      7 of Hearts        King of Diamonds
4 of Clubs         7 of Diamonds      8 of Spades        2 of Spades
3 of Diamonds      7 of Spades        6 of Hearts        9 of Diamonds
Ace of Hearts      8 of Hearts        3 of Hearts        Jack of Spades
Ace of Clubs       10 of Spades       King of Hearts     2 of Diamonds
6 of Diamonds      7 of Clubs         Ace of Diamonds    King of Spades


DEALING 5 CARDS
Jack of Hearts
Queen of Hearts
Queen of Spades
4 of Diamonds
Queen of Diamonds

Displaying card images...

Image display skipped (card_images folder not found).
Error: [Errno 2] No such file or directory: 'C:\\Users\\PAYEL\\card_images\
\Jack_of_Hearts.png'
```

Data Class Advantages over Named Tuples

Data classes offer several advantages over named tuples:

• Although each named tuple technically represents a different type, a named tuple *is a* tuple and *all* tuples can be compared to one another. So, objects of *different* named tuple types could compare as equal if they have the same number of members and the same values for those members. Comparing objects of different data classes *always* returns False, as does comparing a data class object to a tuple object.

• If you have code that unpacks a tuple, adding more members to that tuple breaks the unpacking code. Data class objects cannot be unpacked. So you can add more

data attributes to a data class without breaking existing code.

• A data class can be a base class or a subclass in an inheritance hierarchy.

10.13.4 Data Class Advantages over Traditional Classes

Data classes also offer various advantages over the traditional Python classes you saw earlier in this chapter:
• A data class autogenerates __init__, __repr__ and __eq__, saving you time.

• A data class can autogenerate the special methods that overload the <, <=, > and >= comparison operators.

• When you change data attributes defined in a data class, then use it in a script or interactive session, the autogenerated code updates automatically. So, you have less code to maintain and debug.

• The required variable annotations for class attributes and data attributes enable you to take advantage of static code analysis tools. So, you might be able to eliminate additional errors before they can occur at execution time.

• Some static code analysis tools and IDEs can inspect variable annotations and issue warnings if your code uses the wrong type. This can help you locate logic errors in your code *before* you execute it. In an end-of-chapter exercise, we ask you to use the static code analysis tool MyPy to demonstrate such warnings.

BOOK EXAMPLE:
DATACLASS:
This Python code defines a simple **playing card system** using the @dataclass decorator to manage card attributes efficiently.

```
from dataclasses import dataclass
from typing import ClassVar, List

@dataclass
class Card:
    """Card data class with class attributes, data attributes,
    autogenerated methods and explicitly defined methods."""

    FACES: ClassVar[List[str]] = ['Ace', '2', '3', '4', '5', '6', '7',
                                  '8', '9', '10', 'Jack', 'Queen', 'King']
    SUITS: ClassVar[List[str]] = ['Hearts', 'Diamonds', 'Clubs', 'Spades']

    face: str
    suit: str

    @property
    def image_name(self):
        """Return the Card's image file name."""
```

```python
        return str(self).replace(' ', '_') + '.png'

    def __str__(self):
        """Return string representation for str()."""
        return f'{self.face} of {self.suit}'

    def __format__(self, format_spec):
        """Return formatted string representation."""
        return f'{str(self):{format_spec}}'

class DeckOfCards:
    """Deck of playing cards."""
    def __init__(self):
        """Create deck of 52 Card objects."""
        self._cards = [Card(face, suit) for suit in Card.SUITS for face in Card.FACES]

    def __str__(self):
        """Return string representation of the deck."""
        return ' '.join(f'{card:19}' for card in self._cards)

    def shuffle(self):
        """Shuffle deck of cards."""
        import random
        random.shuffle(self._cards)

    def deal_card(self):
        """Deal one card from the deck."""
        return self._cards.pop() if self._cards else None

# Example Usage
if __name__ == "__main__":
    # Create a card
    c1 = Card(Card.FACES[0], Card.SUITS[3])
    print(c1)  # Ace of Spades
    print(c1.image_name)  # Ace_of_Spades.png

    # Create a deck
    deck = DeckOfCards()
    print(deck)  # Print all cards in order

    # Shuffle deck
    deck.shuffle()
    print(deck)  # Print shuffled deck

    # Deal a card
    dealt_card = deck.deal_card()
    print(f'Dealt card: {dealt_card}')
```

OUTPUT

```
Ace of Spades
Ace_of_Spades.png
Ace of Hearts        2 of Hearts          3 of Hearts          4 of Hearts
5 of Hearts          6 of Hearts          7 of Hearts          8 of Hearts
9 of Hearts          10 of Hearts         Jack of Hearts       Queen of He
arts      King of Hearts       Ace of Diamonds      2 of Diamonds        3
of Diamonds        4 of Diamonds        5 of Diamonds        6 of Diamonds
7 of Diamonds        8 of Diamonds        9 of Diamonds        10 of Diamo
nds       Jack of Diamonds     Queen of Diamonds    King of Diamonds     Ac
e of Clubs         2 of Clubs           3 of Clubs           4 of Clubs
5 of Clubs           6 of Clubs           7 of Clubs           8 of Clubs
9 of Clubs           10 of Clubs          Jack of Clubs        Queen of Cl
ubs       King of Clubs        Ace of Spades        2 of Spades          3
of Spades          4 of Spades          5 of Spades          6 of Spades
7 of Spades          8 of Spades          9 of Spades          10 of Spade
s         Jack of Spades       Queen of Spades      King of Spades
6 of Clubs           5 of Clubs           Queen of Clubs       Jack of Clu
bs        10 of Spades         10 of Clubs          Queen of Spades      3
of Spades          9 of Diamonds        8 of Spades          Ace of Spades
Jack of Diamonds     7 of Diamonds        2 of Hearts          10 of Heart
s         Ace of Hearts        5 of Diamonds        King of Spades       2
of Diamonds        4 of Hearts          8 of Clubs           Queen of Diam
onds    King of Clubs        Queen of Hearts      5 of Spades          7 of
Spades          8 of Diamonds        7 of Clubs           4 of Clubs
9 of Hearts          4 of Diamonds        Jack of Hearts       4 of Spades
Jack of Spades       3 of Diamonds        9 of Spades          8 of Hearts
3 of Hearts          King of Hearts       5 of Hearts          King of Dia
monds     3 of Clubs           10 of Diamonds       6 of Hearts          9
of Clubs           Ace of Diamonds      6 of Spades          Ace of Clubs
7 of Hearts          6 of Diamonds        2 of Spades          2 of Clubs
Dealt card: 2 of Clubs
```

Unit Testing with Docstrings and doctest

Unit testing is a method of testing individual components of a program (such as functions or classes) to ensure they work correctly. The **doctest** module in Python allows us to write test cases inside a function's docstring. These tests can then be executed using doctest.testmod() to verify that the expected results match the actual outputs.

"""Account class definition for demonstrating doctest."""

```
from decimal import Decimal

class Account:
    """Account class for demonstrating doctest."""

    def __init__(self, name, balance):
        """
        Initialize an Account object.

        >>> account1 = Account('John Green', Decimal('50.00'))
        >>> account1.name
```

```
        'John Green'
        >>> account1.balance
        Decimal('50.00')

        The balance argument must be greater than or equal to 0.
        >>> account2 = Account('John Green', Decimal('-50.00'))
        Traceback (most recent call last):
        ...
        ValueError: Initial balance must be >= to 0.00.
        """
        if balance < Decimal('0.00'):
            raise ValueError('Initial balance must be >= to 0.00.')

        self.name = name
        self.balance = balance

    def deposit(self, amount):
        """Deposit money to the account."""
        if amount < Decimal('0.00'):
            raise ValueError('Amount must be positive.')
        self.balance += amount

if __name__ == '__main__':
    import doctest
    doctest.testmod(verbose=True) #runs all tests and prints results.
```

OUTPUT

```
Trying:
    account1 = Account('John Green', Decimal('50.00'))
Expecting nothing
ok
Trying:
    account1.name
Expecting:
    'John Green'
ok
Trying:
    account1.balance
Expecting:
    Decimal('50.00')
ok
Trying:
    account2 = Account('John Green', Decimal('-50.00'))
Expecting:
    Traceback (most recent call last):
    ...
    ValueError: Initial balance must be >= to 0.00.
```

```
ok
18 items had no tests:
    __main__
    __main__.Account
    __main__.Account.deposit
    __main__.Card
    __main__.Card.__format__
    __main__.Card.__init__
    __main__.Card.__repr__
    __main__.Card.__str__
    __main__.Card.face
    __main__.Card.image_name
    __main__.Card.suit
    __main__.DeckOfCards
    __main__.DeckOfCards.__init__
    __main__.DeckOfCards.__str__
    __main__.DeckOfCards.deal_card
    __main__.DeckOfCards.shuffle
    __main__.Student
    __main__.display_cards
1 items passed all tests:
   4 tests in __main__.Account.__init__
4 tests in 19 items.
4 passed and 0 failed.
Test passed.
```